

# Natural-language SQL Agent

An AI agent that turns plain-language questions into SQL, runs them on Postgres, fixes its own mistakes, and answers with an explanation, table, and chart.

## SKILLS SHOWN

Agents · tool use · self-correction · evals

## TOTAL EFFORT

~15–20 working days

## COST

\$0 (managed free tiers)

stack · Next.js (Vercel) · Postgres (Neon free tier) · Gemini Flash / Groq · Vercel AI SDK · recharts

## Milestones — build in order

### 0 Skeleton live

1–2 days

Get a live link on day one — even empty. Proves the deploy pipeline works and builds momentum.

- Next.js app deployed to Vercel (a "hello" page is fine)
- Postgres database created on Neon
- Seed script filling a demo e-commerce schema ( customers , products , orders , order\_items ) with a few thousand realistic rows via @faker-js/faker

**Done when** — a vercel.app link opens, and Neon holds a seeded database.

### 1 Basic text-to-SQL (happy path)

2–3 days

Get the core loop working end-to-end once, no error handling yet — just the straight case.

- Input box where the user types a question
- API route: sends question + DB schema to the LLM, gets a SQL query back
- Run the SQL on Postgres, show the raw result as a simple table

**Done when** — "show the top 5 customers by orders" returns a correct table. No self-correction, no chart, no explanation yet.

## 2 Safety guardrails

1 day

Make the app safe to expose publicly. Small work, but a strong interview point.

- Connect using a **read-only** database role
- Validation layer: block any SQL containing `INSERT / UPDATE / DELETE / DROP / ALTER / TRUNCATE` before execution
- Query timeout so heavy queries can't hang the server

---

**Done when** — a "delete everything" style request is refused gracefully and the data is untouched.

## 3 Self-correction loop

2-3 days

Teach the agent to recover from its own errors. This is what makes it an "agent," not a lookup box.

- On error or empty result, feed the error message back to the LLM: "this SQL failed with this error, fix it"
- Re-run the corrected SQL; retry up to 2-3 times
- If all attempts fail, answer honestly that it couldn't resolve the question

---

**Done when** — a tricky question visibly shows the agent erroring first, then self-correcting to the right answer.

## 4 Answer polish (recruiter-friendly UX)

3-4 days

Make it obvious in the first 10 seconds what the app does. This is where portfolio value lives.

- Plain-language explanation alongside the raw table
- Chart (recharts) when the result suits one
- "Show SQL" toggle for transparency
- Example-question buttons so a visitor knows what to ask

---

**Done when** — someone opens the link, clicks an example, and sees explanation + table + chart with no instructions.

## 5 Eval suite

3–4 days

Prove how reliable the agent is. In 2026 hiring, this is the rarest signal — don't skip it.

- Golden dataset: 30–40 questions with their expected correct answers (JSON)
- Runner script that sends every question through the agent
- Result-based grading: run the agent's SQL and compare its result set to the expected one (no LLM judge needed for most cases)
- Output an accuracy score (e.g. "37/40 = 92.5%")

---

**Done when** — `npm run eval` prints a scorecard of how many questions passed and which failed.

## 6 CI gate

1 day

Make sure no change can quietly make the agent worse.

- GitHub Actions workflow running the eval suite on every pull request
- Fail the build if accuracy drops below a threshold — like a unit test failing

---

**Done when** — intentionally breaking a prompt in a PR turns CI red and blocks the merge.

## 7 Portfolio polish

2 days

Make the work legible at a glance to anyone scanning the repo.

- Clean README: one-line pitch, live link, a GIF demo
- Eval accuracy screenshot at the top — it's what stops the scroll
- Short architecture explanation and an ADR (why result-based evals, why read-only, why a retry limit)
- Pin the repo on GitHub

---

**Done when** — someone understands what it is, how to run it, and why it's well-built from the README alone in 30 seconds.

## Full feature list

### ● Core

- Natural language → SQL translation (schema-aware)
- Live query execution on Postgres
- Self-correction with a retry limit
- Plain-language answer + data table + chart
- "Show SQL" transparency toggle
- Example-question buttons

### ● Reliability & trust

- Result-based eval suite with accuracy score
- CI gate that fails builds on regression
- Golden dataset of 30–40 questions

### ● Safety

- Read-only database connection
- Destructive-SQL blocking (INSERT/UPDATE/DELETE/DROP)
- Query timeout

### ● Portfolio

- One-click live demo, data pre-seeded
- README + GIF + architecture + ADR
- Pinned repo

## How to pace it

- Finish milestones 1–3 first — that's the real "agent." Everything else builds on it.
- Commit and deploy at the end of every milestone so there's always a working live version.
- Target milestones 0–5 for a complete, deployable, eval-backed project (~12–15 days). CI and polish (6–7) can follow.
- Don't skip the eval suite — a RAG project is common; a regression-tested agent is not. That's the differentiator.
- Don't stall on perfection. Meet each "Done when" and move on; polish later.